

Validación, Gestión y Consideraciones Prácticas

Una especificación de requisitos alcanza su valor real solo cuando el equipo puede demostrar que los enunciados acordados describen el sistema correcto y que esa correspondencia se conserva mientras el contexto del proyecto evoluciona. Las tres unidades previas condujeron desde los fundamentos del requisito hasta la producción de una Especificación de Requisitos de Software (ERS) estructurada conforme a la norma *ISO/IEC/IEEE 29148* [10]; este capítulo cierra el ciclo abordando cómo se comprueba la calidad de esos artefactos, cómo se los mantiene bajo control a lo largo del tiempo y con qué herramientas y estándares se gestiona su ciclo de vida en proyectos reales.

El hilo conductor es la transición desde el modelado orientado a la solución hacia el aseguramiento de la calidad. Primero se consolidan los tres tipos de modelos —de datos, funcionales y de comportamiento— que representan la solución desde perspectivas complementarias y que constituyen el objeto concreto sobre el que actuarán las técnicas de validación. A continuación se exponen los principios y las técnicas de validación, la gestión de la naturaleza iterativa del proceso, el control de cambios y la trazabilidad, y finalmente las herramientas CASE y los estándares que sostienen la práctica profesional. La revisión sistemática de Zhao y Zhou [19] confirma que el modelado y la gestión de requisitos, lejos de ser fases aisladas, forman un continuo en el que cada artefacto debe permanecer verificable y trazable.

La discusión se apoya de forma permanente en tres cuerpos normativos y de conocimiento: la norma *ISO/IEC/IEEE 29148* para el proceso y las propiedades de los requisitos, el modelo de calidad de producto *ISO/IEC 25010* [9] para los atributos de calidad que se validan y miden, y la guía SWEBOK v4 [17] como marco de referencia del cuerpo de conocimiento. Sobre esa base se integran resultados recientes de alto impacto que muestran cómo la disciplina incorpora hoy la inteligencia artificial generativa y los modelos de lenguaje de gran tamaño en la validación y la gestión [2, 16].

4.1 Validación de Requisitos

Los requisitos orientados a la solución se documentan mediante modelos que capturan el sistema desde tres perspectivas complementarias: qué información manipula (datos), qué transformaciones realiza (función) y cómo reacciona en el tiempo (comportamiento). Estos modelos son, al mismo tiempo, el resultado del análisis y el insumo principal de la validación, porque un defecto en el modelo se propaga a la implementación con un costo de corrección creciente [14]. Por ello, antes de estudiar las técnicas con que se comprueban, conviene precisar la semántica de cada tipo de modelo y las relaciones que los vinculan, tema que ocupa las cuatro subsecciones siguientes.

4.1.1 Modelos de datos: Entidad-Relación (ER) y diagramas de clases UML

La perspectiva de datos describe las entidades del dominio, sus atributos y las asociaciones entre ellas, sin comprometerse todavía con decisiones de implementación. El modelo Entidad-Relación propuesto por Chen [1] sigue siendo la notación canónica para esta perspectiva: representa entidades como rectángulos, relaciones como rombos y atributos como elipses, y anota la cardinalidad (1:1, 1:N, M:N) sobre las líneas de asociación. La Figura 4.1 muestra un fragmento de modelo ER para un dominio de pedidos, en el que un **Cliente** realiza cero o más **Pedidos**.

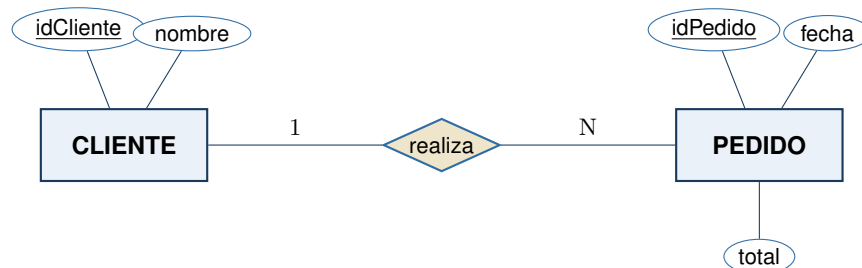


Figura 4.1. Fragmento de modelo Entidad-Relación para un dominio de pedidos, con atributos clave subrayados y cardinalidad 1:N.

El diagrama de clases del Lenguaje Unificado de Modelado (UML) generaliza al modelo ER al incorporar el comportamiento en la propia estructura. Según la especificación *OMG UML 2.5.1* [13], cada clase se representa con un rectángulo dividido en tres compartimentos —nombre, atributos y operaciones— y las asociaciones se anotan con multiplicidades en sus extremos. La Figura 4.2 traduce el modelo ER anterior a un diagrama de clases: las columnas de atributos preservan la información del dominio, mientras que las operaciones añaden la semántica funcional que el modelo ER no expresa. Esta equivalencia parcial es la que permite derivar esquemas de base de datos a partir de diagramas de clases y viceversa.



Figura 4.2. Diagrama de clases UML equivalente al modelo ER de la Figura 4.1; los compartimentos de operaciones añaden la semántica funcional.

La elección entre ambas notaciones no es indiferente para la validación. El modelo ER favorece la revisión con expertos del dominio poco familiarizados con la programación, porque su vocabulario es cercano al del negocio; el diagrama de clases, en cambio, facilita la verificación de consistencia con el diseño orientado a objetos y con las reglas de integridad. Wieggers y Beatty [18] recomiendan mantener ambos niveles sincronizados mediante un diccionario de datos único, de modo que cada entidad, atributo y relación tenga una definición textual precisa que sirva de referencia en las inspecciones.

4.1.2 Modelos funcionales: diagramas de flujo de datos, DFD esencial y actividades UML

La perspectiva funcional describe cómo el sistema transforma entradas en salidas mediante procesos que consumen y producen flujos de datos. El Diagrama de Flujo de Datos (DFD), formalizado en el análisis estructurado de DeMarco [4], emplea cuatro símbolos: procesos, flujos de datos, almacenes y entidades externas. La Figura 4.3 presenta un DFD de nivel 1 para

el registro de un pedido, donde el proceso 1.0 **Registrar pedido** recibe los datos de la entidad externa **Cliente**, deja constancia en el almacén **D1 Pedidos** y devuelve una confirmación.

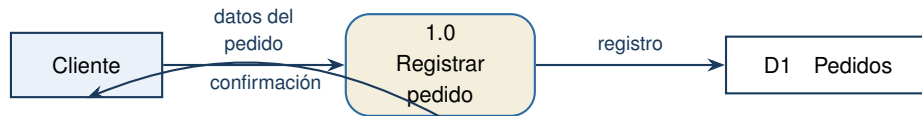


Figura 4.3. Diagrama de Flujo de Datos de nivel 1 para el registro de un pedido; los almacenes se etiquetan con el prefijo D.

El DFD *esencial* lleva esta abstracción al límite: describe la lógica del sistema suponiendo una tecnología perfecta —procesadores infinitamente rápidos y memoria ilimitada—, de modo que quedan únicamente las transformaciones y los almacenes que el dominio exige, libres de detalles de implementación. Esta depuración es valiosa para la validación porque separa los requisitos esenciales de las decisiones de diseño, y permite preguntar a los interesados si la lógica capturada es correcta con independencia de cómo se implemente. Pohl [14] advierte, no obstante, que el DFD carece de una semántica de control precisa, por lo que debe complementarse con modelos de comportamiento.

El diagrama de actividades UML es la notación moderna que subsana esa carencia, al integrar flujo de control y flujo de datos con soporte explícito para decisiones, bifurcaciones, concurrencia y carriles de responsabilidad. La Figura 4.4 modela el mismo proceso de pedido incorporando una decisión sobre la disponibilidad de *stock*, con dos caminos alternativos que convergen en un nodo final. Frente al DFD, el diagrama de actividades expresa el orden temporal de las acciones, lo que facilita validar reglas de negocio como “no se confirma un pedido sin verificar existencias”.

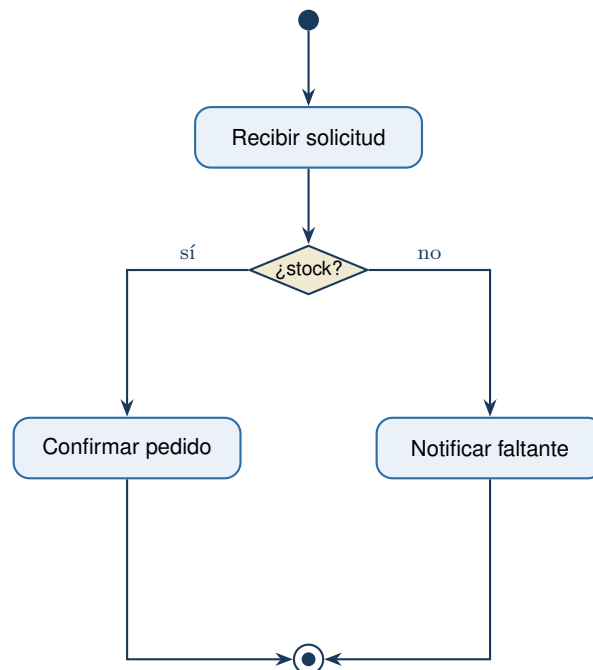


Figura 4.4. Diagrama de actividades UML del registro de un pedido; la decisión *¿stock?* gobierna dos caminos que convergen en el nodo final.

4.1.3 Modelos de comportamiento: autómatas finitos, statecharts y máquinas de estado UML

La perspectiva de comportamiento describe cómo cambia el estado del sistema —o de una entidad relevante— en respuesta a los eventos que recibe. El autómata finito es el modelo básico: un conjunto finito de estados, un estado inicial, transiciones etiquetadas con eventos y, en su versión reconocedora, uno o más estados de aceptación. La Figura 4.5 modela el ciclo de vida de un **Pedido** como una máquina de estados UML, notación que hereda del autómata finito pero admite condiciones de guarda, acciones de entrada/salida y estados compuestos.

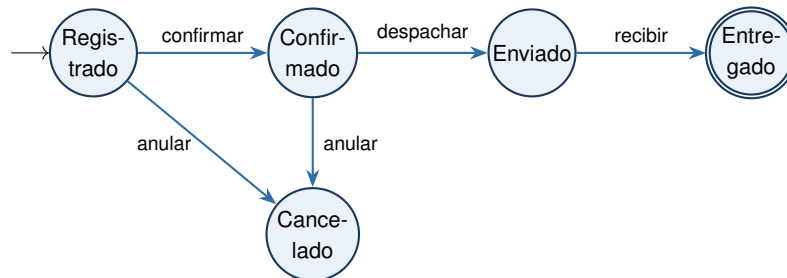


Figura 4.5. Máquina de estados UML del ciclo de vida de un pedido; **Entregado** es el estado final de aceptación.

Cuando el número de estados crece, el autómata plano se vuelve inmanejable. Los *statecharts* de Harel [7] introducen tres mecanismos que dominan esa complejidad: la jerarquía (un superestado agrupa subestados y factoriza las transiciones comunes), la ortogonalidad (regiones concurrentes que describen comportamiento paralelo) y la memoria de historia. La Figura 4.6 ilustra la jerarquía: el superestado **Activo** agrupa a **Confirmado** y **Enviado**, de modo que la transición **anular** se define una sola vez a nivel del superestado en lugar de repetirse en cada subestado. UML adoptó directamente este formalismo en sus máquinas de estado, lo que convierte a los statecharts en el sustrato teórico de la notación de comportamiento más usada en la industria.

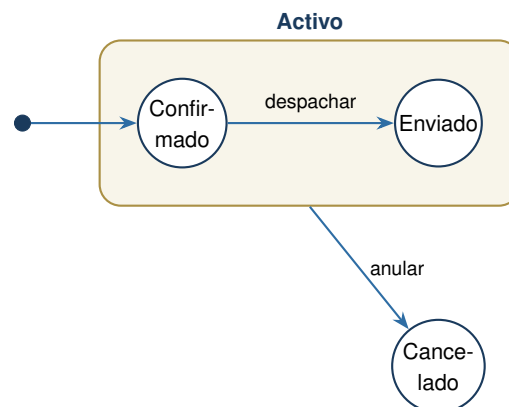


Figura 4.6. Statechart con jerarquía: el superestado **Activo** factoriza la transición **anular** de sus subestados.

4.1.4 Interrelaciones entre las tres perspectivas: datos, función y comportamiento

Ninguna de las tres perspectivas basta por sí sola para especificar un sistema no trivial, porque cada una silencia información que las otras hacen explícita. El modelo de datos ignora el orden temporal; el modelo funcional no fija el ciclo de vida de las entidades; el modelo de comportamiento presupone unos datos y unas funciones que no define. La Figura 4.7 representa esta complementariedad como un triángulo cuyos vértices se conectan mediante relaciones de consistencia que la validación debe verificar de forma cruzada.

La consistencia entre perspectivas es una fuente sistemática de defectos que las inspecciones

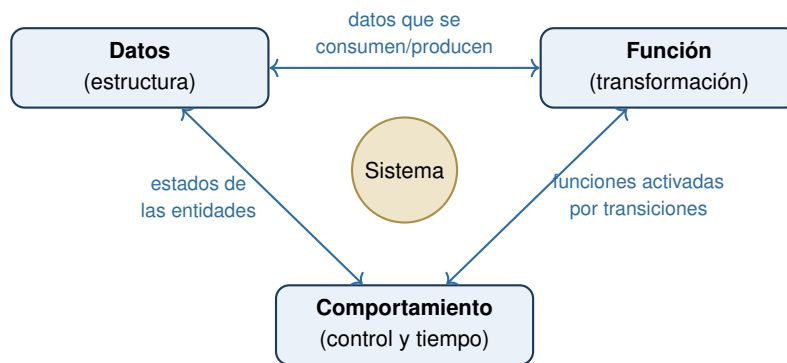


Figura 4.7. Complementariedad de las tres perspectivas; las aristas indican las relaciones de consistencia que la validación verifica de forma cruzada.

deben rastrear. Un evento que aparece en la máquina de estados debe corresponder a alguna función del modelo funcional; toda entidad citada por una función debe existir en el modelo de datos; y cada atributo modificable debe tener asociado algún cambio de estado observable cuando el dominio así lo exige. La revisión sistemática de Zhao y Zhou [19] sobre ingeniería de requisitos basada en modelos documenta que la mayoría de las técnicas maduras persiguen precisamente esta integración, y que las herramientas de transformación de modelos permiten hoy verificar automatizadamente varias de estas reglas de correspondencia.

Desde la óptica de la calidad, esta triangulación es una forma económica de validación: al contrastar cada perspectiva con las otras dos se detectan omisiones y contradicciones que una lectura aislada no revelaría, en aplicación directa del principio de validación desde distintas vistas que se formaliza en la sección siguiente. La norma *ISO/IEC/IEEE 29148* [10] refuerza esta idea al exigir que el conjunto de requisitos sea internamente consistente, propiedad que solo puede comprobarse examinando las relaciones entre los modelos y no cada modelo por separado.

4.2 Gestión de Requisitos

Comprobar que los modelos anteriores describen el sistema correcto exige una disciplina propia, con motivación, principios y técnicas diferenciadas. La validación de requisitos es la actividad que examina los artefactos de requisitos para descubrir defectos antes de que se propaguen al diseño y a la codificación, donde corregirlos cuesta órdenes de magnitud más [14, 15]. Las cuatro subsecciones siguientes recorren su fundamento —los principios y el aseguramiento de la calidad—, las técnicas de evaluación basadas en lectura humana, el uso de prototipos y la validación de modelos, metas y escenarios, y finalmente las pruebas de aceptación como puente hacia la verificación del producto.

4.2.1 Motivación y seis principios de validación. Aseguramiento de la calidad en IR

La motivación de la validación es económica y a la vez técnica: un requisito erróneo que sobrevive hasta la fase de pruebas puede costar hasta cien veces más que si se hubiera detectado durante la elicitación [15]. El aseguramiento de la calidad en ingeniería de requisitos verifica tres aspectos complementarios que Pohl [14] denomina las tres dimensiones de calidad: el *contenido* (que todos los requisitos relevantes estén presentes y sean correctos), la *documentación* (que se ajusten a las reglas de formato y estilo acordadas) y el *acuerdo* (que los interesados hayan consensuado cada requisito). La Figura 4.8 representa estas tres dimensiones y su intersección, que corresponde a un requisito plenamente validado.

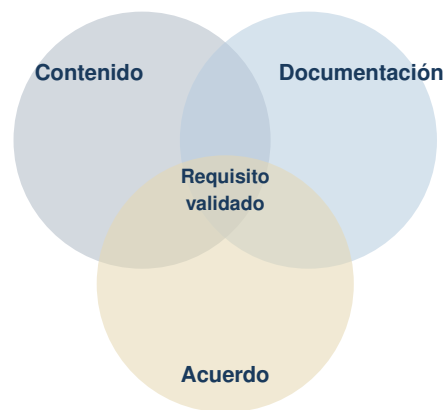


Figura 4.8. Las tres dimensiones de calidad del aseguramiento en IR; su intersección define un requisito validado.

Sobre esa base, Pohl [14] formula seis principios que guían toda actividad de validación con independencia de la técnica empleada. La Tabla 4.1 los resume. El segundo principio —separar la identificación de la corrección— es especialmente relevante en la práctica, porque mezclar ambas actividades en una misma reunión desvía la atención hacia la discusión de soluciones y reduce la tasa de defectos detectados; por ello las inspecciones formales prohíben explícitamente proponer correcciones durante la sesión de detección.

Tabla 4.1. Los seis principios de validación de requisitos según Pohl [14].

Nº	Principio	Idea central
1	Involucrar a los interesados correctos	Solo quien posee el conocimiento del dominio puede juzgar si un requisito es correcto.
2	Separar identificación y corrección de errores	Detectar defectos y corregirlos son actividades distintas; mezclarlas reduce la eficacia.
3	Validar desde distintas vistas	Contrastar el mismo requisito bajo perspectivas de datos, función y comportamiento revela inconsistencias.
4	Cambiar de tipo de documentación	Reexpresar un requisito en otra representación (texto a modelo, o viceversa) expone ambigüedades ocultas.
5	Construir artefactos de desarrollo	Prototipos y modelos ejecutables hacen tangibles los defectos que el texto oculta.
6	Validar de forma repetida	La validación no es un hito único sino una actividad recurrente a lo largo del proceso.

El aseguramiento de la calidad moderno complementa estos principios con apoyo automatizado. La revisión sistemática de Cheng et al. [2] documenta que los modelos de lenguaje de gran tamaño ya se aplican con resultados prometedores a la detección de ambigüedad, la comprobación de completitud y la generación de preguntas de validación, aunque advierte que su salida requiere supervisión humana porque puede introducir afirmaciones plausibles pero incorrectas. Esta cautela es coherente con el primer principio: la responsabilidad de juzgar la corrección sigue recayendo en los interesados y el equipo, no en la herramienta.

4.2.2 Evaluación de requisitos: revisiones informales, walkthroughs e inspecciones formales (Fagan)

Las técnicas de evaluación basadas en lectura humana forman un continuo de formalidad creciente. La *revisión informal* consiste en que uno o más colegas comenten un documento sin procedimiento predefinido; es barata y útil en etapas tempranas, pero su cobertura es irregular. El *walkthrough* añade estructura: el autor guía a los participantes a través del artefacto para recoger observaciones, aunque conserva un carácter formativo y poco medido. La *inspección formal* de Fagan [5] es el extremo riguroso del continuo, con roles definidos, criterios de entrada y salida, y recolección de métricas de defectos. La Tabla 4.2 contrasta las tres técnicas.

Tabla 4.2. Comparación de las técnicas de evaluación de requisitos por lectura.

Criterio	Revisión informal	Walkthrough	Inspección (Fagan)
Formalidad	Baja	Media	Alta
Roles definidos	No	Parcial (autor guía)	Sí (moderador, autor, lector, inspectores, registrador)
Criterios de entrada/salida	No	No	Sí
Métricas de defectos	No	Ocasional	Sí, sistemáticas
Costo relativo	Bajo	Medio	Alto

La inspección de Fagan se organiza en seis fases encadenadas que la Figura 4.9 representa. Tras la *planificación* (seleccionar el material y el equipo) y una *visión general* opcional que nivela el conocimiento, cada inspector realiza una *preparación* individual buscando defectos; la *reunión de inspección* los registra sin corregirlos —en aplicación del segundo principio de validación—; el autor ejecuta el *retrabajo*; y el moderador comprueba en el *seguimiento* que todo defecto se haya resuelto, decidiendo si el artefacto pasa o debe reinspeccionarse.

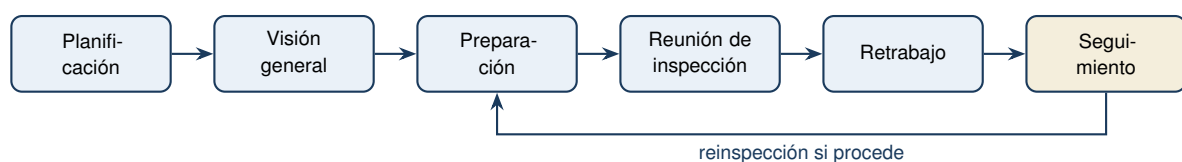


Figura 4.9. Las seis fases de la inspección formal de Fagan [5]; el seguimiento puede reactivar la preparación.

La evidencia acumulada muestra que las inspecciones formales son una de las técnicas de aseguramiento de la calidad con mejor relación costo-beneficio, capaces de eliminar una fracción elevada de los defectos antes de la codificación [15]. Su limitación es el esfuerzo que demandan, por lo que en la práctica se reservan para los artefactos de mayor riesgo —requisitos críticos de seguridad o de alto impacto contractual— y se combinan con revisiones más ligeras para el resto del documento.

4.2.3 Prototipos como técnica de validación. Validación del modelo. Validación de metas y escenarios

El quinto principio de validación —construir artefactos de desarrollo— encuentra su expresión más directa en el prototipado. Un prototipo hace tangible una parte del sistema y permite que los interesados reaccionen ante algo concreto, revelando necesidades latentes que ninguna lectura de texto habría expuesto. La Figura 4.10 organiza los prototipos según tres criterios que

Pohl [14] y Sommerville [15] distinguen: el objetivo, el alcance y el destino final del artefacto.

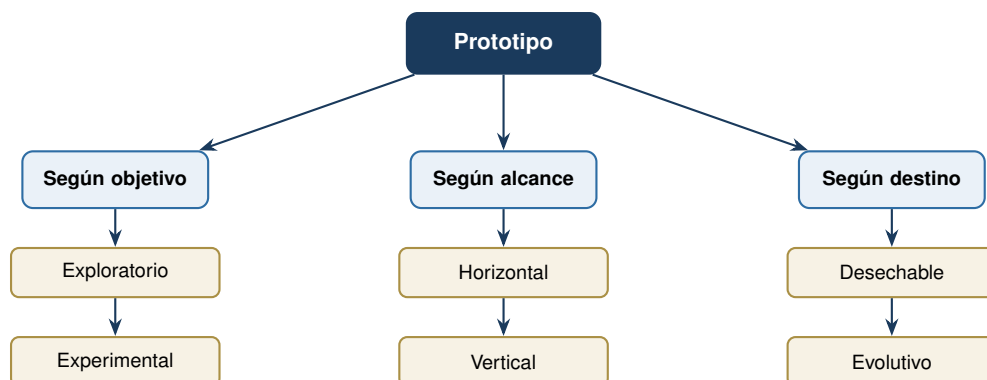


Figura 4.10. Clasificación de prototipos según objetivo, alcance y destino, usados como técnica de validación.

Más allá del prototipo, la validación del modelo comprueba que los diagramas producidos en la Sección 4.1 sean correctos respecto de la realidad que pretenden describir, y consistentes entre sí. Aquí el cuarto principio —cambiar de tipo de documentación— resulta decisivo: traducir un requisito textual a una máquina de estados obliga a explicitar el estado inicial, las transiciones y las condiciones de guarda, y en ese proceso afloran huecos que el enunciado en prosa toleraba. Recorrer la máquina de estados de la Figura 4.5 junto a un experto del dominio es, en sí mismo, un acto de validación.

La validación de metas y escenarios cierra el cuadro comprobando que la solución especificada satisface las intenciones de alto nivel de los interesados. Se verifica que cada meta se descomponga en subrequisitos que efectivamente la realizan, y que los escenarios —casos de uso o historias de usuario— cubran tanto los caminos normales como las excepciones relevantes. Wiegers y Beatty [18] recomiendan derivar de cada escenario un conjunto de condiciones observables que servirán, más adelante, como base de las pruebas de aceptación, enlazando así la validación de requisitos con la verificación del producto terminado.

4.2.4 Pruebas de aceptación: definición, tipos (funcional, no funcional, regresión) y relación con requisitos

La prueba de aceptación es la comprobación formal, ejecutada con la participación o en representación del cliente, de que el sistema entregado satisface los requisitos acordados y es apto para su uso. Se distingue de la verificación interna en su propósito y en su audiencia: no busca defectos de implementación sino evidencia de que el producto correcto fue construido, y su criterio de éxito son los requisitos, no el diseño. Por ello cada prueba de aceptación debe trazar a uno o más requisitos, como ilustra la Figura 4.11, y un requisito sin prueba asociada es una señal de requisito no verificable —defecto que la norma *ISO/IEC/IEEE* 29148 clasifica como violación de la propiedad de verificabilidad [10].

Según el atributo de calidad que examinan, las pruebas de aceptación se agrupan en tres tipos. Las *funcionales* comprueban que el sistema produce las salidas correctas ante entradas dadas, es decir, que cumple los requisitos funcionales. Las *no funcionales* verifican atributos como rendimiento, seguridad, usabilidad o fiabilidad, en correspondencia con las características del modelo de calidad de producto *ISO/IEC* 25010 [9]. Las de *regresión* reejecutan pruebas previamente superadas tras un cambio, para asegurar que la nueva versión no rompió funcionalidad ya aceptada; su papel es central en la gestión de cambios que se aborda en la sección siguiente, pues cada modificación de un requisito obliga a revalidar los requisitos dependientes.

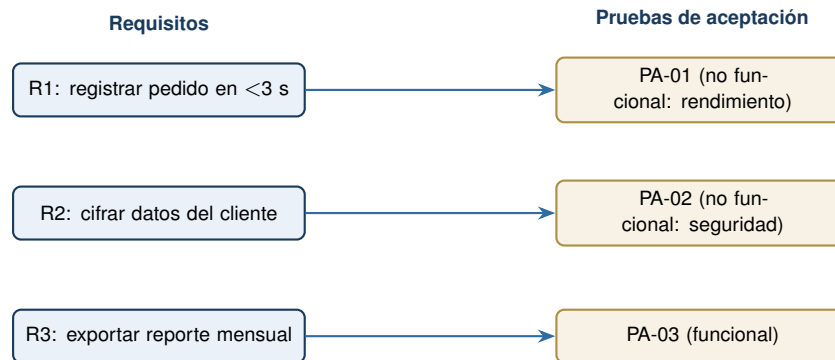


Figura 4.11. Cada prueba de aceptación traza al menos a un requisito; la ausencia de traza delata un requisito no verificable.

El aseguramiento de la trazabilidad entre requisitos y pruebas de aceptación es, en consecuencia, un requisito de gestión y no meramente de prueba. La guía SWEBOK v4 [17] sitúa la definición de criterios de aceptación dentro de las buenas prácticas de la ingeniería de requisitos, precisamente porque son el instrumento que hace operativa la propiedad de verificabilidad y conecta la validación temprana con la aceptación final del producto.

4.3 Herramientas, Estándares e IR Ágil

Validar los requisitos una vez no basta: el contexto de un proyecto cambia, los interesados revisan sus necesidades y los artefactos deben mantenerse coherentes mientras evolucionan. La gestión de requisitos es la actividad transversal que preserva esa coherencia a lo largo del tiempo, controlando la naturaleza iterativa del proceso, los cambios, la trazabilidad y la configuración de los artefactos [10, 14]. Las cuatro subsecciones siguientes desarrollan estos pilares, que constituyen la base sobre la que operan las herramientas CASE y los estándares tratados en la última sección del capítulo.

4.3.1 Naturaleza iterativa del proceso de IR. Gestión del contexto y de actividades de IR

La ingeniería de requisitos no es una fase que concluye antes del diseño, sino un proceso iterativo cuyas actividades se entrelazan y se repiten a medida que crece el conocimiento del problema. Pohl [14] organiza este proceso en tres actividades centrales —elicitación, documentación y negociación— y dos actividades transversales —validación y gestión— que actúan de forma continua sobre las primeras. La Figura 4.12 representa esta estructura: las actividades centrales forman un ciclo que se recorre tantas veces como sea necesario, mientras que la validación y la gestión lo envuelven de manera permanente.

Gestionar este proceso implica gestionar también el contexto que lo rodea. El límite del sistema, los interesados, las normas aplicables y las decisiones ya tomadas conforman un contexto que no es estático: un cambio regulatorio o la incorporación de un nuevo interesado pueden invalidar requisitos que se creían estables. Por ello la gestión de actividades de IR planifica qué técnicas aplicar en cada iteración, asigna responsabilidades y decide cuándo un artefacto está suficientemente maduro para congelarse en una línea base. Hoy y Xu [8] muestran, en su revisión sistemática sobre IR ágil, que esta gestión iterativa del contexto es precisamente el punto donde los enfoques ágiles aportan mayor valor, al reemplazar la especificación exhaustiva inicial por refinamientos sucesivos guiados por la retroalimentación.

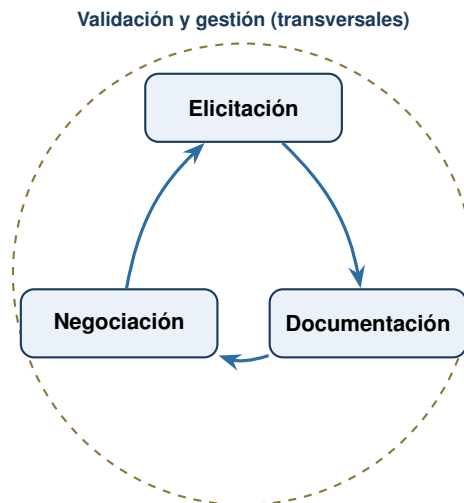


Figura 4.12. Naturaleza iterativa del proceso de IR: las tres actividades centrales forman un ciclo envuelto por la validación y la gestión.

4.3.2 Gestión del cambio: Change Control Board, documentación de cambios y proceso de cambio

Todo requisito consolidado en una línea base queda protegido por un proceso de control de cambios que impide modificaciones arbitrarias y garantiza que cada alteración sea evaluada, aprobada y documentada. El órgano responsable de esas decisiones es el *Change Control Board* (CCB), un comité que reúne a representantes de los interesados, del equipo técnico y de la gestión del proyecto, con autoridad para aprobar, rechazar o diferir cada solicitud de cambio [18]. La Figura 4.13 describe el flujo típico de una solicitud de cambio desde su registro hasta su cierre.

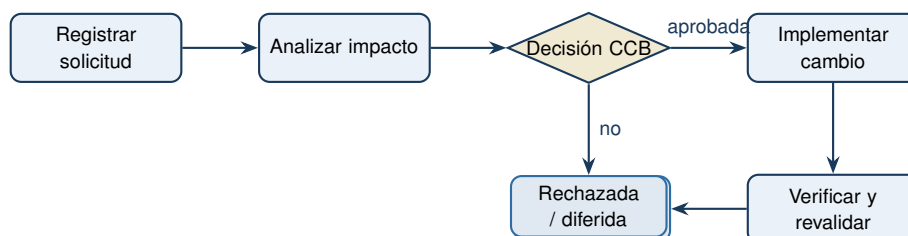


Figura 4.13. Proceso de control de cambios: la decisión del CCB bifurca hacia la implementación o el rechazo, y todo cambio aprobado se verifica y documenta.

La documentación del cambio es tan importante como la decisión misma. Cada solicitud debe registrar su origen, la justificación, el análisis de impacto sobre requisitos y artefactos dependientes, la decisión adoptada con su fundamento y el estado de implementación. Esta bitácora cumple una función doble: sostiene la rendición de cuentas —quién aprobó qué y por qué— y alimenta la trazabilidad, pues el análisis de impacto solo es fiable si existen enlaces explícitos entre el requisito modificado y los elementos que dependen de él. La norma *ISO/IEC/IEEE 29148* [10] exige, en efecto, que los requisitos sean gestionables, propiedad que presupone un proceso de cambio documentado como el descrito.

4.3.3 Seguimiento y trazabilidad: tres tipos, documentación, visualización y trazabilidad dirigida por uso

La trazabilidad es la capacidad de seguir la vida de un requisito en ambos sentidos: hacia sus orígenes y hacia los artefactos que se derivan de él. A partir de la distinción seminal de Gotel y

Finkelstein [6] entre trazabilidad previa y posterior a la especificación, se reconocen tres tipos que la Tabla 4.3 sintetiza y la Figura 4.14 representa: la trazabilidad hacia atrás, que vincula cada requisito con las fuentes que lo motivan; la trazabilidad hacia adelante, que lo conecta con el diseño, el código y las pruebas; y la trazabilidad horizontal, que registra las dependencias entre requisitos.

Tabla 4.3. Los tres tipos de trazabilidad de requisitos.

Tipo	Vincula	Uso principal
Hacia atrás (pre-especificación)	Requisito → fuentes, interesados, normas	Justificar el porqué de cada requisito y valorar cambios de origen.
Hacia adelante (post-especificación)	Requisito → diseño, código, pruebas	Comprobar cobertura e impacto de cambios sobre la solución.
Horizontal (entre requisitos)	Requisito ↔ requisito	Detectar dependencias, conflictos y redundancias.

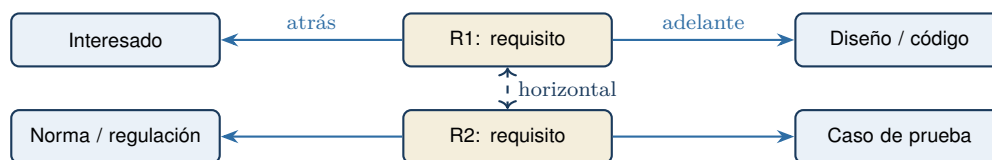


Figura 4.14. Grafo de trazabilidad: enlaces hacia atrás (a fuentes), hacia adelante (a solución) y horizontales (entre requisitos).

La documentación de la trazabilidad se apoya habitualmente en la *matriz de trazabilidad*, que cruza requisitos con artefactos y marca cada enlace en la celda correspondiente; su lectura por filas revela requisitos huérfanos sin implementación, y por columnas, artefactos sin requisito que los justifique. Para conjuntos grandes, la visualización como grafo dirigido resulta más legible que la matriz, y las herramientas CASE permiten navegar interactivamente esos enlaces. No obstante, mantener trazabilidad completa es costoso, por lo que Cleland-Huang et al. [3] proponen la *trazabilidad dirigida por uso*: capturar solo los enlaces que responderán a preguntas concretas previstas —análisis de impacto, cobertura de pruebas, cumplimiento normativo—, de modo que el esfuerzo de trazado se justifique por su utilidad. La revisión de Mucha et al. [11] confirma que la trazabilidad hacia las fuentes sigue siendo la más descuidada en la práctica, pese a ser la que más valor aporta cuando cambian las necesidades de origen.

4.3.4 Requerimientos de medición. Priorización. Gestión de configuración y baselines

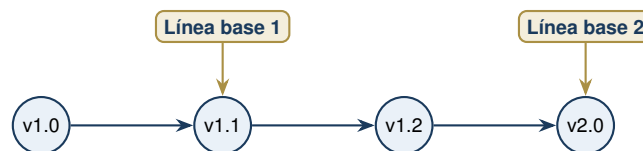
Lo que no se mide no se puede gestionar. La medición en IR observa atributos como el número de requisitos por estado, la volatilidad —proporción de requisitos modificados por unidad de tiempo—, la densidad de defectos detectados en inspección o la cobertura de trazabilidad, y usa esos indicadores para anticipar riesgos: una volatilidad alta y sostenida, por ejemplo, señala que el problema aún no se comprende y que congelar una línea base sería prematuro. La guía SWEBOK v4 [17] recomienda definir cada métrica a partir de la pregunta de gestión que pretende responder, evitando medir por inercia.

La priorización ordena los requisitos según su valor y su urgencia para decidir qué se implementa primero, decisión crítica cuando los recursos son limitados. La Tabla 4.4 resume el método MoSCoW, ampliamente usado tanto en contextos tradicionales como ágiles por su simplicidad y su capacidad de forzar acuerdos explícitos entre interesados [18].

Tabla 4.4. Categorías de priorización MoSCoW.

Categoría	Significado para la planificación
<i>Must have</i>	Imprescindible; sin él la entrega carece de valor o de viabilidad legal.
<i>Should have</i>	Importante pero no vital; se incluye si el tiempo lo permite.
<i>Could have</i>	Deseable; primera reserva a sacrificar ante presión de plazos.
<i>Won't have (this time)</i>	Fuera del alcance actual; se documenta para una entrega futura.

La gestión de configuración cierra el ciclo al identificar de forma unívoca cada versión de cada artefacto y al agrupar versiones coherentes en *líneas base* (baselines): conjuntos de requisitos revisados y aprobados que sirven de referencia estable para el desarrollo y que solo pueden modificarse a través del proceso de control de cambios de la Subsección 4.3.2. La Figura 4.15 muestra la evolución versionada de una ERS con dos líneas base sucesivas. Sin este mecanismo, el equipo perdería la referencia de “qué se acordó construir”, y la trazabilidad y las métricas dejarían de tener un ancla fiable.

**Figura 4.15.** Versionado de una ERS con dos líneas base; cada baseline congela un conjunto aprobado modificable solo vía control de cambios.

4.4 Ingeniería de Requisitos en Procesos Ágiles

Los principios de gestión de la sección anterior se materializan en la práctica mediante herramientas de ingeniería asistida por computadora (CASE) y estándares que fijan qué propiedades debe exhibir un requisito bien gestionado. Esta sección examina el instrumental de la disciplina: las herramientas que soportan la elicitación, el modelado y la gestión; los estándares que guían su aplicación; los esquemas de atributos que hacen operativa la gestión y la trazabilidad; y la gestión de configuración del documento de requisitos como artefacto vivo.

4.4.1 Herramientas CASE para IR: Jira, Trello, Enterprise Architect, Sparx EA y ReqIF

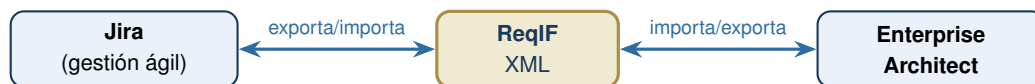
El ecosistema de herramientas para IR abarca desde gestores ligeros de tareas hasta plataformas de modelado de nivel empresarial. Jira soporta la gestión de *backlogs*, historias de usuario y flujos de trabajo configurables, y domina en equipos ágiles; Trello ofrece tableros Kanban de baja fricción, adecuados para proyectos pequeños o para la fase inicial de organización de ideas. En el extremo del modelado, Enterprise Architect —producto de la empresa Sparx Systems, por lo que “Sparx EA” designa a la misma herramienta— integra el modelado UML y SysML con la gestión de requisitos y la trazabilidad en un repositorio único. La Tabla 4.5 sitúa estas herramientas según su categoría y su aporte a la IR.

ReqIF merece una mención aparte porque no es una herramienta sino un estándar de intercambio. El formato *Requirements Interchange Format* de la *Object Management Group* [12] define una representación XML común que permite transferir requisitos —con sus atributos, valores y enlaces de trazabilidad— entre herramientas de distintos proveedores, resolviendo el problema de la fragmentación de la cadena de suministro del software. La Figura 4.16 ilustra su papel

Tabla 4.5. Herramientas de apoyo a la ingeniería de requisitos y el formato de intercambio ReqIF.

Herramienta	Categoría	Aporte principal a la IR
Jira	Gestión ágil de trabajo	Backlog, historias de usuario, flujos y métricas de avance.
Trello	Tableros Kanban	Organización visual ligera de requisitos y tareas.
Enterprise Architect (Sparx EA)	Modelado CASE integral	Modelado UML/SysML, atributos y trazabilidad en un repositorio.
ReqIF	Formato de intercambio	Intercambio de requisitos entre herramientas sin pérdida de atributos ni enlaces.

como lengua franca entre una herramienta de gestión ágil y una de modelado.

**Figura 4.16.** ReqIF como formato neutral de intercambio de requisitos entre herramientas de distinto propósito y proveedor.

La incorporación de inteligencia artificial a este instrumental es la tendencia más reciente. Stein et al. [16] describen cómo los modelos de lenguaje de gran tamaño pueden integrarse en el proceso de requisitos para asistir en la redacción, la clasificación y la detección de inconsistencias, actuando como una capa de apoyo sobre las herramientas CASE existentes en lugar de reemplazarlas. La revisión de Hoy y Xu [8] matiza, sin embargo, que en entornos ágiles el valor de la herramienta depende de que se integre con el flujo de trabajo del equipo, y no al revés.

4.4.2 Estándares IEEE 29148:2018 e ISO/IEC 25010: aplicación práctica en proyectos reales

Los estándares traducen los principios de la disciplina en criterios verificables. La norma *ISO/IEC/IEEE 29148:2018* [10] fija las propiedades que debe cumplir un requisito individual —necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto y conforme— y las de un conjunto de requisitos —completo, consistente, asequible y acotado—. En la práctica, estas propiedades se operacionalizan como listas de comprobación de las inspecciones de la Subsección 4.2.2: cada criterio se convierte en una pregunta que el inspector formula sobre cada requisito.

Complementariamente, el modelo de calidad de producto *ISO/IEC 25010:2023* [9] proporciona el vocabulario para especificar y validar los requisitos no funcionales. Su revisión de 2023 organiza la calidad del producto en nueve características —frente a las ocho de la versión de 2011—, tras añadir *Seguridad física* (Safety) y redefinir la antigua Usabilidad como *Capacidad de interacción* y la Portabilidad como *Flexibilidad*. La Figura 4.17 presenta las nueve características, que en un proyecto real actúan como una taxonomía para no omitir dimensiones de calidad relevantes.

La aplicación conjunta de ambos estándares en un proyecto real sigue un patrón reconocible: 29148 gobierna la forma y las propiedades de los enunciados, mientras que 25010 aporta el catálogo de atributos de calidad que esos enunciados deben cuantificar. Así, un requisito no funcional bien formado nombra una característica de 25010, fija una métrica y un umbral, y satisface los criterios de verificabilidad de 29148, de modo que pueda derivarse de él una prueba de aceptación no funcional como las descritas en la Subsección 4.2.4.

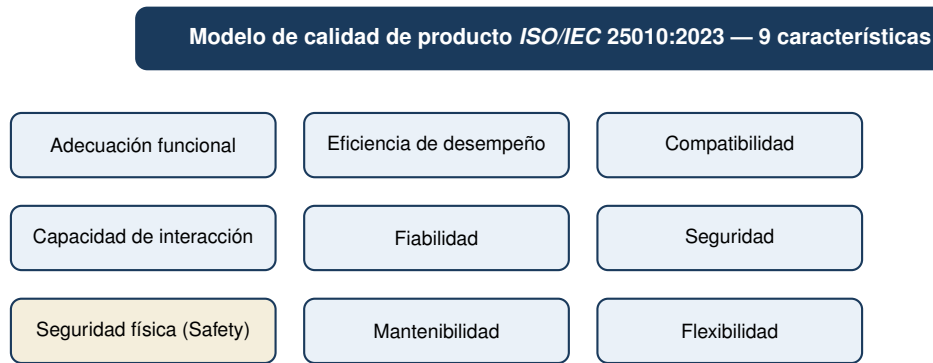


Figura 4.17. Las nueve características del modelo de calidad de producto *ISO/IEC 25010:2023*; en gris, la incorporada en la revisión de 2023.

4.4.3 Atributos de requisitos para gestión. Esquemas de atributos y trazabilidad en herramientas CASE

Gestionar un requisito exige tratarlo no como una frase aislada sino como un registro con atributos. Un *esquema de atributos* define qué información acompaña a cada requisito —identificador, prioridad, estado, fuente, estabilidad, versión, método de verificación y responsable, entre otros—, y es la estructura que las herramientas CASE materializan como campos de una ficha. La Tabla 4.6 propone un esquema mínimo alineado con la norma *ISO/IEC/IEEE 29148* [10] y con las recomendaciones de Wiegers y Beatty [18].

Tabla 4.6. Esquema mínimo de atributos para la gestión de un requisito.

Atributo	Propósito de gestión
Identificador	Referencia unívoca y estable para trazabilidad y control de cambios.
Prioridad	Ordena la implementación (por ejemplo, categoría MoSCoW).
Estado	Situación en el ciclo de vida: propuesto, aprobado, implementado, verificado.
Fuente	Origen del requisito, base de la trazabilidad hacia atrás.
Estabilidad	Probabilidad de cambio; anticipa riesgo y volatilidad.
Versión	Habilita la gestión de configuración y las líneas base.
Método de verificación	Inspección, análisis, demostración o prueba; asegura verificabilidad.

Sobre este esquema se apoya la trazabilidad gestionada en la herramienta. Cada campo de fuente materializa un enlace hacia atrás, y cada vínculo con casos de prueba o elementos de diseño, uno hacia adelante, de modo que la herramienta puede computar automáticamente el análisis de impacto de un cambio recorriendo el grafo de la Figura 4.14. La calidad de este análisis depende directamente de la disciplina con que se mantengan los atributos: un estado desactualizado o una fuente omitida degradan la trazabilidad hasta volverla engañosa, riesgo que la trazabilidad dirigida por uso de la Subsección 4.3.3 busca acotar concentrando el esfuerzo donde rinde.

La ingeniería de requisitos basada en modelos lleva este planteamiento un paso más allá, al almacenar los requisitos y sus enlaces como elementos de un modelo formal susceptible de análisis automático. La revisión sistemática de Zhao y Zhou [19] evidencia que este enfoque mejora la consistencia y la trazabilidad, aunque a costa de una mayor inversión inicial en la definición del metamodelo y en la formación del equipo.

4.4.4 Gestión de configuración del ERS: versiones, configuraciones y baselines de artefactos

El documento de Especificación de Requisitos de Software no es un entregable inmóvil sino un artefacto que evoluciona, y gestionar su configuración significa controlar esa evolución de manera reproducible. Conviene distinguir tres conceptos que suelen confundirse: la *versión* es un estado identificado de un único artefacto en el tiempo; la *configuración* es un conjunto coherente de versiones de varios artefactos que funcionan juntos —la ERS, sus modelos UML, la matriz de trazabilidad y los casos de prueba—; y la *línea base* es una configuración revisada, aprobada y congelada que sirve de referencia. La Figura 4.18 representa una configuración que agrupa versiones específicas de cuatro artefactos y se etiqueta como línea base.

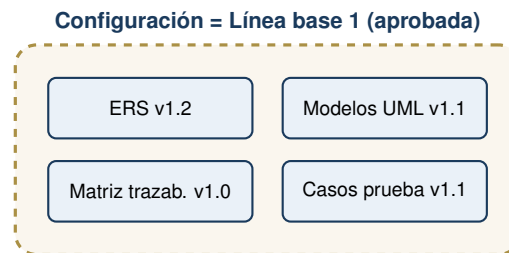


Figura 4.18. Una configuración agrupa versiones coherentes de varios artefactos; al aprobarse y congelarse se convierte en línea base.

La gestión de configuración del ERS integra así todos los mecanismos del capítulo. Una línea base solo se modifica a través del proceso de control de cambios y su CCB (Subsección 4.3.2); cada modificación preserva la trazabilidad (Subsección 4.3.3) y actualiza los atributos de versión (Subsección 4.4.3); y la comparación entre líneas base sucesivas alimenta las métricas de volatilidad (Subsección 4.3.4). La guía SWEBOK v4 [17] subraya que esta integración es la que confiere reproducibilidad al proyecto: ante cualquier versión entregada, el equipo puede reconstruir con exactitud qué requisitos, modelos y pruebas la sustentaban.

Cerrando el capítulo, conviene notar que la validación y la gestión estudiadas aquí no son etapas terminales sino disciplinas transversales que acompañan todo el ciclo de vida. La calidad de una ERS no se decide en un hito final de revisión, sino que se construye de forma incremental mediante la validación repetida, el control disciplinado de los cambios y la trazabilidad sostenida, apoyados en estándares y herramientas que hacen operativos esos principios en proyectos reales.

Referencias

- [1] Peter Pin-Shan Chen. The entity-relationship model — toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976. doi: 10.1145/320434.320440.
- [2] Haowei Cheng, Jati H. Husen, Yijun Lu, Teeradaj Racharak, Nobukazu Yoshioka, Naoyasu Ubayashi, and Hironori Washizaki. Generative AI for requirements engineering: A systematic literature review. *Software: Practice and Experience*, 56(2):141–170, 2026. doi: 10.1002/spe.70029.
- [3] Jane Cleland-Huang, Orlena C. Z. Gotel, Jane Huffman Hayes, Patrick Mäder, and Andrea Zisman. Software traceability: Trends and future directions. In *Proceedings of the Future of Software Engineering (FOSE 2014)*, pages 55–69, Hyderabad, India, 2014. ACM. doi: 10.1145/2593882.2593891.
- [4] Tom DeMarco. *Structured Analysis and System Specification*. Prentice Hall, Englewood Cliffs, NJ, USA, 1979. ISBN 978-0-13-854380-6.
- [5] Michael E. Fagan. Design and code inspections to reduce errors in program development. *IBM Systems Journal*, 15(3):182–211, 1976. doi: 10.1147/sj.153.0182.
- [6] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. An analysis of the requirements traceability problem. In *Proceedings of the IEEE International Conference on Requirements Engineering (ICRE'94)*, pages 94–101, Colorado Springs, CO, USA, 1994. IEEE. doi: 10.1109/ICRE.1994.292398.
- [7] David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987. doi: 10.1016/0167-6423(87)90035-9.
- [8] Zoe Hoy and Mark Xu. Agile software requirements engineering challenges-solutions — a conceptual framework from systematic literature review. *Information*, 14(6):322, 2023. doi: 10.3390/info14060322.
- [9] ISO/IEC. ISO/IEC 25010:2023. systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model. International Standard, 2023.
- [10] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018. systems and software engineering — life cycle processes — requirements engineering. International Standard, 2018.
- [11] Julia Mucha, Andreas Kaufmann, and Dirk Riehle. A systematic literature review of pre-requirements specification traceability. *Requirements Engineering*, 29(2):119–141, 2024. doi: 10.1007/s00766-023-00412-z.
- [12] Object Management Group. Requirements interchange format (ReqIF), version 1.2. OMG Standard, document formal/2016-07-01, 2016.
- [13] Object Management Group. Unified modeling language (OMG UML), version 2.5.1. OMG Standard, document formal/2017-12-05, 2017.

- [14] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Berlin, Heidelberg, Germany, 2010. ISBN 978-3-642-12577-5.
- [15] Ian Sommerville. *Software Engineering*. Pearson, Harlow, England, 10th edition, 2016. ISBN 978-0-13-394303-0.
- [16] Armin Stein, Aksir Mirzai, Joachim Axmann, and Thomas Vietor. Integrating the capabilities offered by large language models into the requirements engineering process. *Digital Engineering*, 10:100098, 2026. doi: 10.1016/j.dte.2026.100098.
- [17] Hironori Washizaki, editor. *Guide to the Software Engineering Body of Knowledge (SWE-BOK Guide), Version 4.0*. IEEE Computer Society, Los Alamitos, CA, USA, 2024.
- [18] Karl Wieggers and Joy Beatty. *Software Requirements*. Microsoft Press, Redmond, WA, USA, 3rd edition, 2013. ISBN 978-0-7356-7966-5.
- [19] Zhongpei Zhao and Zhengshu Zhou. A systematic literature review on model-based requirements engineering. *Journal of Systems and Software*, 237:112836, 2026. doi: 10.1016/j.jss.2026.112836.